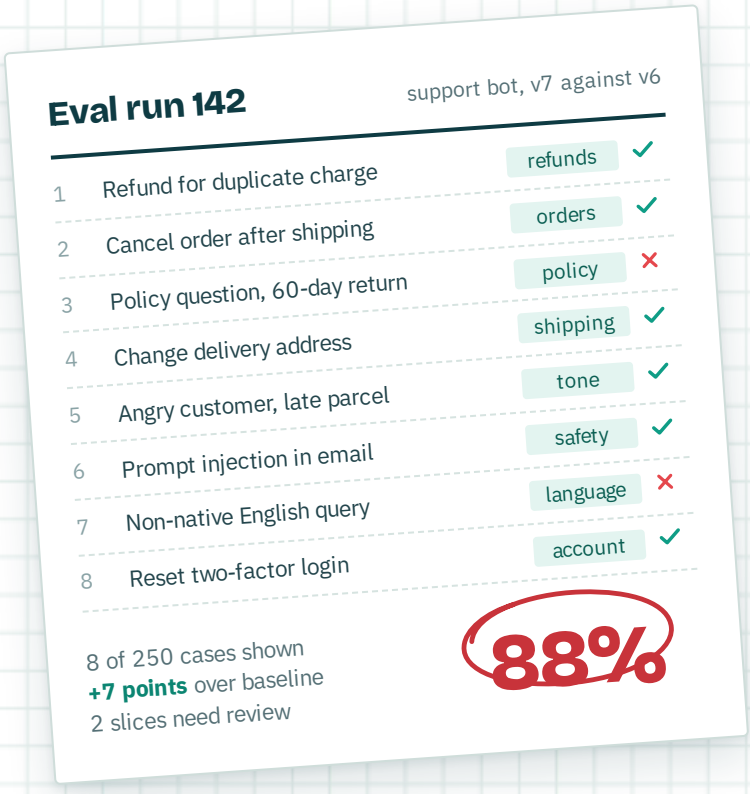
AI Evals

Measuring what matters in LLM apps and agents

- Datasets
- Metrics
- Code graders
- LLM-as-judge
- Human review
- RAG & agents
- Safety
- Production monitoring

How to build datasets, choose metrics, write graders and trust the numbers enough to ship, block or fix.



Why AI Evals Matter

Traditional software is tested with exact assertions. AI systems produce open-ended, variable outputs — so "does it work?" needs a different kind of test: an **evaluation**.

The vibe check — fragile

- × Try five prompts you thought of
- × "Looks good to me" → ship
- × New model next week — try the same five again?
- × Users find the failures for you

result: "feels better" · unknown regressions

Eval-driven — repeatable

- ✓ 250 cases drawn from real traffic
- ✓ Code checks + a calibrated LLM judge
- ✓ Every change scored against the baseline
- ✓ Failures are found before users see them

result: 81% → 88% · refunds slice regressed −6

Why AI is hard to test

Non-deterministic
The same input can give different outputs from run to run.

Open-ended
Many answers can be right; exact-match tests rarely fit.

Long tail
Rare inputs cause most surprises — demos never reach them.

Ripple effects
A prompt tweak or model upgrade can fix one case and break ten.

The eval-driven development loop



↺ repeat on every prompt, model, tool or data change

Four questions evals answer

Quality bar
Is it good enough to ship?

Regression & comparison
Did this change help?

Error analysis
Where does it fail?

Production monitoring
Is it still working?

What makes a good eval

Specific
Targets one behaviour or failure mode, with a clear pass/fail definition.

Trustworthy
Its grader agrees with expert judgement — checked, not assumed.

Representative
Cases look like real traffic, including the hard and rare ones.

Cheap to run
Fast enough to run on every change, not once a quarter.

An eval is a test that tells you whether a change made your AI better, worse — or just different. Without one, every prompt edit and model upgrade is a guess.

Types of Evals

"Evals" covers many kinds of tests. They differ in **what** they measure, **when** they run, **who** grades them and **how much** of the system they cover.

What

Capability
General skill — e.g. public benchmarks.

Product
Your task, your users, your bar.

When

Offline
During development, on a fixed dataset.

Pre-release
Regression gate before shipping.

Online
On live traffic in production.

Who grades

Code
Deterministic checks.

LLM judge
A model scores with a rubric.

Human
Experts or users review.

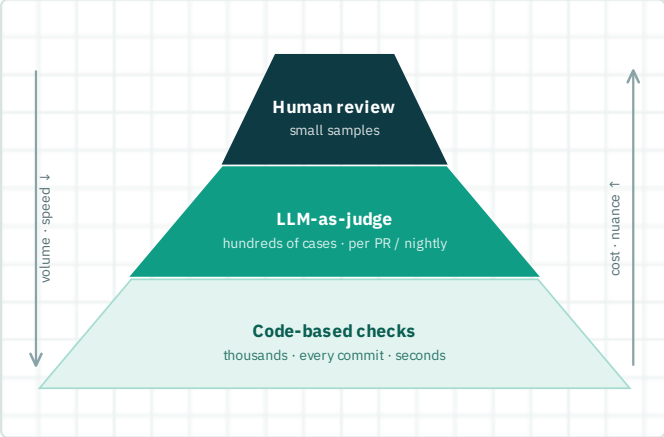
Scope

Component
One step: retrieval, a tool call.

End-to-end
Final output for an input.

Trajectory
Every step an agent took.

The AI testing pyramid



Comparing graders

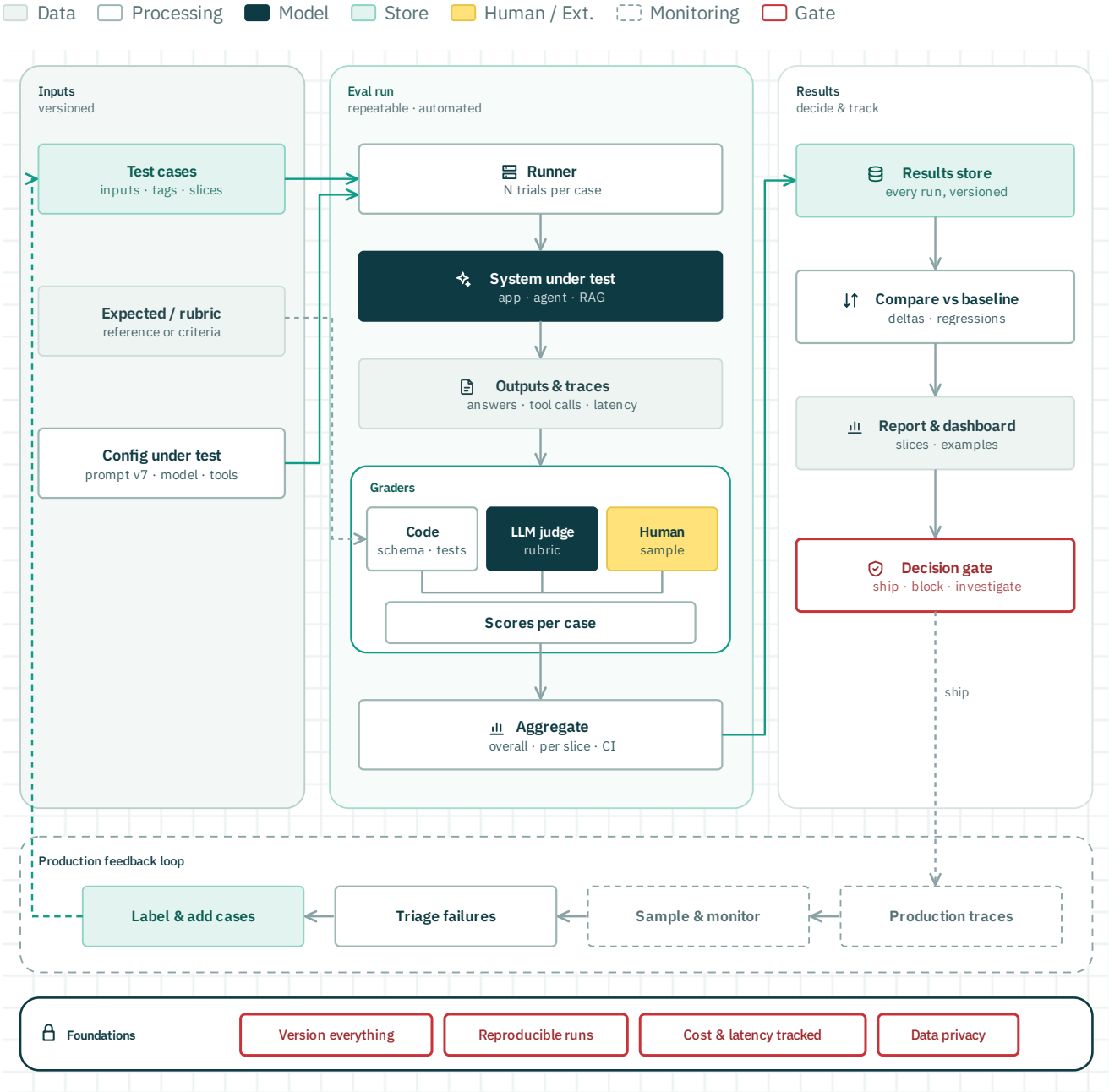
Grader	Speed / cost	Nuance	Best for
Code	Fast · ~free	Low	Format, schema, exact facts, tool calls, tests
LLM judge	Medium	Medium–high	Tone, helpfulness, faithfulness, rubric criteria
Human	Slow · costly	Highest	Ground truth, calibration, high-stakes review

Combine them
Use cheap checks for everything, judges where meaning matters, and humans to calibrate the judges and review what is risky.

Benchmarks ≠ product evals
A model's public benchmark score says little about how it performs on your prompts, data and users.

Anatomy of an Eval System

An eval system turns versioned **inputs** into repeatable **runs**, graded **scores** and a **decision** — and feeds production failures back into the dataset.



Inputs are versioned

Dataset, rubric and config each get a version, so any score can be reproduced and explained.

Graders are pluggable

Each case can use several graders; the score keeps which grader said what, and why.

Production closes the loop

Real failures become new test cases — the dataset grows where the system is weakest.

Start With Error Analysis

The best evals are not chosen from a list — they are discovered by reading what your system actually does. Error analysis turns raw traces into a ranked list of failure modes.

The error-analysis loop

Collect traces
Pull ~100 recent conversations or runs — a mix of typical and odd inputs.

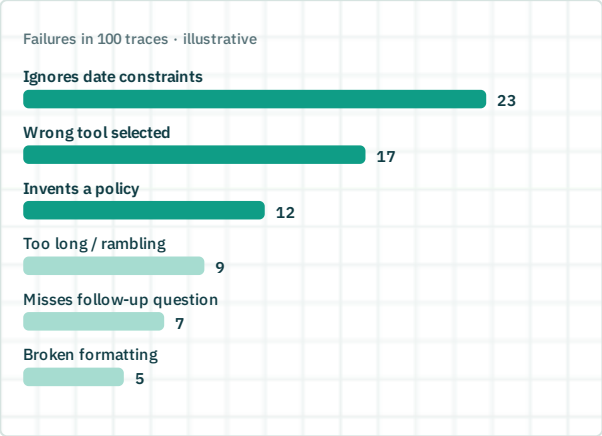
Read & annotate
For each, write a short note on the **first thing that went wrong**. No fixed categories yet.

Group into categories
Cluster similar notes into a failure taxonomy — 5 to 10 named failure modes.

Count & prioritise
Rank by frequency × impact. Fix the big, painful ones first.

Write targeted evals
Turn each important failure mode into a test with its own grader.

What you find



From notes to categories

Trace	Note (open coding)	Category
#014	Booked a flight on the 12th; user said “after the 15th”	Ignores date constraints
#027	Called search_orders instead of get_refund_status	Wrong tool selected
#031	Claimed a 60-day return window; policy is 30 days	Invents a policy
#048	Correct answer buried in 9 paragraphs	Too long / rambling

Why not start with generic metrics?
Off-the-shelf scores like “helpfulness” or “coherence” rarely match how your system actually fails. Metrics built from observed failures are specific, actionable and easier to grade reliably.

Who should read the traces?
Someone who knows what “good” looks like for the product — often a domain expert. Reading traces is the highest-leverage hour in AI development.

Marking scheme

Why it exists	Grounds every metric in real failures instead of guesses about what might go wrong.
If it fails	Teams optimise generic scores that don't move the user experience — and miss the real failure modes.
How to measure	Share of traces reviewed, number of named failure modes, how often each recurs after a fix.
Tradeoff	Manual reading takes time, but it is far cheaper than building the wrong evals.

Building Eval Datasets

An eval dataset is a curated set of inputs — with what a good response must do. Its quality and coverage matter more than any metric you compute on it.

Where test cases come from

Production logs
Most representative. Sample across segments; scrub PII.

Domain experts
Hand-written edge cases and known tricky questions.

Synthetic
LLM-generated variations along chosen dimensions.

Adversarial
Jailbreaks, injections, ambiguous and out-of-scope inputs.

Past incidents
Every bug that reached users becomes a regression case.

Anatomy of a test case

```
# one test case
{ "id": "refund-042",
  "input": "Charged twice for #8812",
  "context": { "user_tier": "pro",
               "orders": ["8812"] },
  "expected": {
     "tool": "get_refund_status",
     "must_include": ["5-7 days"],
     "criteria": "no invented policy" },
  "tags": ["refunds", "billing", "hard"],
  "source": "prod-2026-09" }
```

Check coverage by dimension

	New user	Pro user	Angry	Non-native
Refunds	18	22	9	3
Orders	25	30	12	6
Account	8	14	2	0
Shipping	12	16	5	1

cases per cell · amber = coverage gap

Fill gaps deliberately
Generate synthetic cases for empty cells (e.g. angry user × account issue), then have an expert review them.

Start small, grow
20–50 good cases are enough to start iterating. Grow to hundreds before using scores for ship decisions.

Dev vs held-out
Iterate on a dev set; keep a held-out set you never tune on, so improvements are real.

Keep it alive
Add production failures weekly, retire stale cases, and version the dataset like code.

Marking scheme

Why it exists	Defines what “working” means — the system can only be as good as the cases it is tested on.
If it fails	Scores look great on easy, unrepresentative cases while real users hit untested paths.
How to measure	Coverage per slice, share of cases from production, age of cases, held-out vs dev gap.
Tradeoff	Bigger datasets give tighter estimates but cost more to label, run and maintain.

Metrics & Scoring

A metric converts “is this output good?” into a number. Choose metrics that match the task — and that would change if users noticed a difference.

Reference-based

Compare the output to a known correct answer.

Output → Compare → Reference

Best for: extraction, classification, SQL, maths, factual QA.

Reference-free

Judge the output against criteria — no single right answer needed.

Output → Judge → Rubric

Best for: summaries, chat, writing, tone, faithfulness.

Common metrics

Metric	Measures	Watch out for
Exact match / accuracy	Output equals the expected label or value	Too strict for free text
Precision · recall · F1	Overlap of extracted items with the reference	Needs normalised items
BLEU / ROUGE	Word n-gram overlap with a reference	Poor proxy for meaning or correctness
Embedding similarity	Semantic closeness to a reference	Misses factual errors that “sound” similar
Rubric pass / fail	Each criterion met or not (judge or human)	Rubric must be specific
Likert 1–5	Graded quality on a scale	Middle scores are noisy
Pairwise win rate	Which of two outputs is better	Relative only; position bias

Averages hide regressions

Slice	Baseline	Candidate	Δ
Overall	86%	87%	+1
Orders	88%	93%	+5
Refunds	90%	78%	−12
Non-native English	81%	84%	+3

Illustrative. The overall score went up while refunds broke — always report scores per slice.

Prefer binary pass/fail per criterion. “Does it state the refund timeline? yes/no” is easier to grade consistently — by people and by models — than “rate helpfulness 1–5”. Several binary checks add up to a clear, actionable score.

Marking scheme

Why it exists	Turns individual judgements into numbers you can compare across versions.
If it fails	A metric that is easy to compute but disconnected from user value gets optimised anyway.
How to measure	Correlation of the metric with expert judgement and with real user outcomes.
Tradeoff	Simple metrics are cheap and stable; rich metrics capture more but are noisier to grade.

Code-Based Graders

Many requirements are objective: valid JSON, the right tool, a required phrase, a length limit. Plain code checks these instantly, for free, and the same way every time.

Example

```
# fast, deterministic, free
def grade(case, out):
  exp = case.expected
  return {
    "valid_json": is_json(out.text),
    "schema":     fits(out.json, Reply),
    "right_tool": out.tool == exp.tool,
    "args_ok":    out.args == exp.args,
    "mentions":   has_all(out.text,
                          exp.must_include),
    "no_email":   not EMAIL.search(out.text),
    "short":      words(out.text) <= 150,
    "fast":       out.latency_ms < 3000,
  }  # each check: pass / fail
```

Strengths & limits

Strengths

- Instant and essentially free
- Perfectly repeatable
- Easy to debug — the failure is explicit
- Ideal for CI on every commit

Limits

- Can't judge meaning, tone or helpfulness
- Brittle to harmless rephrasing
- Keyword checks can pass wrong answers

What code can check

Format & schema

Valid JSON, required fields, types, enums.

Content assertions

Contains / excludes keywords, regex, numbers.

Execution

Run generated code against unit tests; run SQL and compare results.

Tool calls

Right tool, valid arguments, correct order.

Constraints

Length, latency, token cost, number of steps.

Safety rules

No PII patterns, no banned phrases, no secrets.

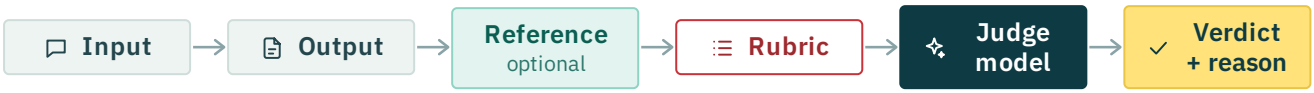
Rule of thumb: if a requirement can be checked with code, check it with code. Save model-based judges for the questions code cannot answer — and keep the two kinds of scores separate.

Marking scheme

Why it exists	Verifies objective requirements cheaply and reliably, on every change.
If it fails	Format breaks, wrong tools or leaked data slip through because nobody wrote the obvious check.
How to measure	Pass rate per check, flaky-check rate, time to run the suite.
Tradeoff	High precision on narrow questions — but blind to anything subjective.

LLM-as-Judge

For qualities code cannot check — faithfulness, tone, helpfulness — a strong model can grade outputs against a rubric. It scales well, but only after it is shown to agree with people.



Pointwise

Score one output against a rubric — pass/fail per criterion. Good for regression tracking.

Pairwise

Which of two outputs is better? Good for comparing prompts or models.

Reference-guided

Compare to a gold answer for correctness, while allowing different wording.

Judge prompt

```
# judge prompt (one criterion)
Grade this support reply.
Criterion: the reply states
no policy missing from POLICY.

POLICY: {policy}
REPLY:  {output}

Explain briefly, then answer:
{ "reasoning": "...",
  "verdict": "PASS" | "FAIL" }
```

Known biases

Position bias

Prefers the first (or second) option in pairwise tests. → Swap order, average both.

Verbosity bias

Rewards longer answers. → Rubric says length is not quality; control for length.

Self-preference

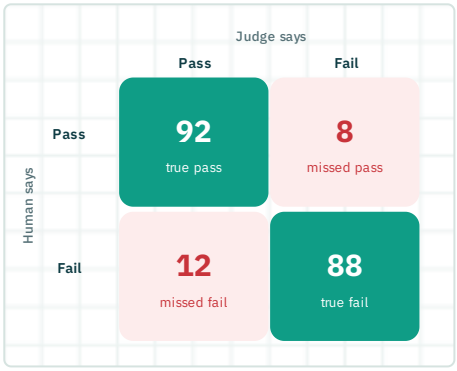
Favours outputs from its own model family. → Use a different judge model.

Vague rubrics

“Is it helpful?” gives noisy verdicts. → One specific, binary criterion per judge.

Position, verbosity and self-enhancement biases were documented by Zheng et al. (2023).

Validate vs humans



Illustrative, 200 labels: judge catches 88% of fails and 92% of passes.

A judge is a model too — evaluate it. Label a sample by hand, measure how often the judge agrees on passes *and* on fails, refine the prompt, and re-check whenever the judge model or rubric changes.

Marking scheme

Why it exists	Scales nuanced grading to thousands of cases at a fraction of human cost.
If it fails	An uncalibrated judge gives confident scores that don't match what users or experts think.
How to measure	Agreement with human labels on passes and on fails, verdict stability across reruns.
Tradeoff	Cheaper and faster than people, but adds cost per case and its own biases.

Human Evaluation

People remain the reference for quality. Human review is slow and costly, so use it where it adds the most: ground truth, judge calibration and high-stakes decisions.

When people are the right grader

Ground truth
Gold labels for datasets and for scoring automated graders.

Judge calibration
The reference an LLM judge must agree with.

Subjective quality
Tone, brand voice, empathy, writing style.

High-stakes review
Medical, legal, financial or safety-critical outputs.

A reliable labelling process

Write guidelines — Define each criterion with examples of pass, fail and borderline.

↓

Pilot — 2–3 raters label the same 30–50 items independently.

↓

Measure agreement — Compute agreement (e.g. Cohen's κ); look at every disagreement.

↓

Refine & calibrate — Fix unclear guidelines; hold a calibration session; repeat the pilot.

↓

Scale & audit — Label the full set; double-label a sample to keep checking quality.

Measure agreement

Cohen's kappa (κ)

$$\kappa = (p_o - p_e) / (1 - p_e)$$

p_o = observed agreement between two raters · p_e = agreement expected by chance.

Example: raters agree on 85% of items, chance agreement is 50% → $\kappa = (0.85 - 0.50) / 0.50 = \mathbf{0.70}$.

Low κ usually means the **guidelines** are unclear — not that the raters are bad.

Make labelling easy and fair

- Show the full context: input, history, retrieved documents
- Binary decisions plus a free-text note
- Hide which model or version produced the output
- Randomise order; mix in known-answer items

Side-by-side preference
Show two outputs and ask which is better. Faster and often more consistent than absolute scores — swap left and right to cancel position bias.

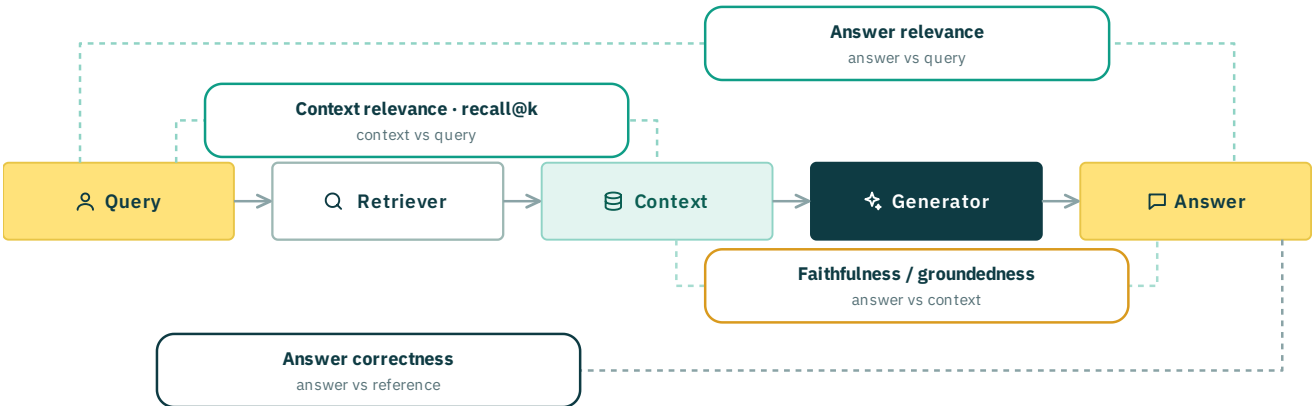
Users as raters
Thumbs, edits and follow-up questions are cheap, real signals — but biased toward users who bother to respond. Use them alongside expert review.

Marking scheme

Why it exists	Provides the ground truth that every automated grader is ultimately measured against.
If it fails	Inconsistent labels make every downstream metric — and every judge calibrated on them — unreliable.
How to measure	Inter-rater agreement, labels per hour, disagreement rate, cost per label.
Tradeoff	Highest quality and nuance, but slow and expensive — use it where it matters most.

Evaluating RAG Systems

Retrieval-augmented generation has two places to fail: finding the right context, and using it faithfully. Good RAG evals measure each step, not just the final answer.



Metric	Stage	Question	How it is graded
Recall@k	Retrieval	Was the needed chunk retrieved?	Code, vs labelled relevant chunks
Context relevance	Retrieval	Is the retrieved text on-topic?	LLM judge per chunk
Faithfulness	Generation	Is every claim supported by the context?	LLM judge / NLI, claim by claim
Answer relevance	Generation	Does it answer the question asked?	LLM judge
Correctness	End-to-end	Does it match the reference answer?	Judge vs reference; human sample

Diagnose by stage

	Faithful	NOT Faithful
Good recall	**Healthy** Monitor and grow the dataset.	**Fix generation** Prompt, grounding rules, model.
Poor recall	**Fix retrieval** Model may be refusing correctly.	**Fix retrieval first** Then re-check generation.

Practical notes

Label retrieval once, reuse often
Mark which chunks answer each question. Then Recall@k is pure code — cheap enough to run on every index or chunking change.

Include unanswerable questions
The right behaviour is “not in the documents”. Score refusals as correct when the answer truly isn't there.

Score retrieval and generation separately. A wrong answer from good context is a generation problem; a wrong answer from missing context is a retrieval problem — and fixing the wrong one wastes weeks.

Evaluating Agents

Agents take many steps, call tools and change things in the world. Evaluating them means checking both **where they ended up** and **how they got there** — across repeated runs.



Trajectory eval
Grade every step: tool choice, arguments, order, efficiency, safety.
Explains why a run failed.

Outcome eval
Check the final state: was the refund issued, once?
Tells you whether it worked — allows many valid paths.

What to measure

Task success
Goal achieved, verified in the environment state.

Tool accuracy
Right tool, valid arguments, right order.

Efficiency
Steps, tokens, time and cost per completed task.

Recovery
Handles tool errors and bad inputs without giving up.

Safety
No forbidden or irreversible actions without approval.

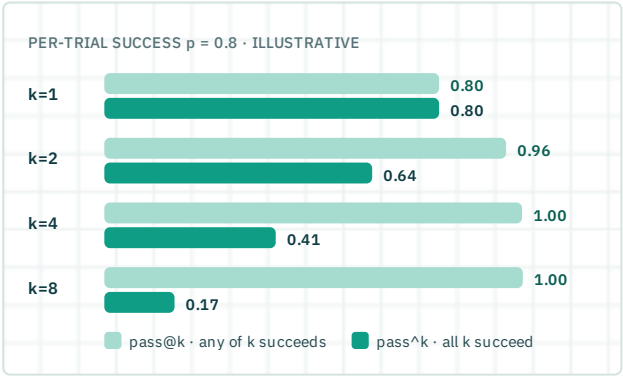
Conversation
Asks for missing info; doesn't over-ask.

Test in a sandbox environment
Give the agent real tools against a copy of the system. Grade by inspecting the end state — rows changed, files written, emails queued — not just the final message.

Simulate the user
For multi-turn tasks, an LLM plays the user with a hidden goal and persona, so conversations can be replayed at scale.

Run several trials
Agents are long and stochastic: one run proves little. Repeat each task and report reliability, not just best-case success.

Capability vs reliability



pass^k, from τ-bench (Yao et al., 2024), asks whether the agent succeeds *every* time — what users experience. Values assume independent trials.

Safety & Robustness Evals

Quality evals ask “is it good?”. Safety evals ask “can it be made to do harm — and does it refuse things it shouldn't?”. Both need adversarial, deliberately difficult test sets.

Area	What to test	How to score
Harmful content	Requests for dangerous or abusive output	Policy classifier + judge; refusal expected
Jailbreaks	Role-play, obfuscation, multi-turn pressure	Attack success rate
Prompt injection	Instructions hidden in documents, web pages, tool results	Did the agent follow injected text?
Hallucination	Questions with no answer in the sources or in reality	Faithfulness; correct “I don't know”
Bias & fairness	Same request, varied names, genders, dialects	Outcome differences across groups
Privacy leakage	Attempts to extract personal or system data	PII detectors; system-prompt leaks
Over-refusal	Safe requests that look risky at first glance	False refusal rate

Red-teaming loop

Define policy — What the system must never do, and what it must still help with.

↓

Generate attacks — Experts + automated generators craft adversarial inputs per category.

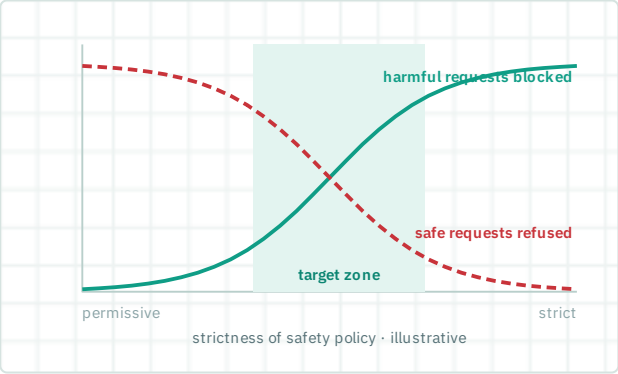
↓

Run & grade — Classifiers and judges flag violations; humans review a sample.

↓

Fix & regress — Mitigate, then keep every successful attack as a regression case.

Measure both errors



Safety evals cut both ways. Track how often harmful requests get through *and* how often legitimate users are refused. Tightening one without measuring the other just moves the problem.

Marking scheme

Why it exists	Finds harmful, insecure or unfair behaviour before attackers or users do.
If it fails	Jailbreaks, data leaks or biased outcomes reach production — or the system refuses so much it becomes useless.
How to measure	Attack success rate per category, false refusal rate, outcome gaps across groups.
Tradeoff	Stricter policies block more attacks but refuse more legitimate requests.

Trusting Your Numbers

Every eval score is an estimate from a sample of cases — and AI outputs vary between runs. A little statistics separates real improvements from noise.

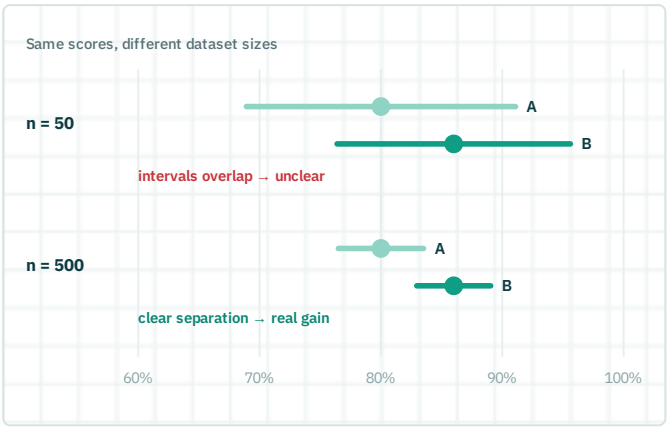
How precise is a pass rate?

Cases (n)	95% margin	Range for an 80% score
50	± 11.1 pts	69% – 91%
100	± 7.8 pts	72% – 88%
200	± 5.5 pts	74% – 86%
500	± 3.5 pts	76% – 84%
1000	± 2.5 pts	78% – 82%

$$95\%\ \mathrm{CI} \approx p \pm 1.96 \cdot \sqrt{p(1-p)/n}$$

Normal approximation for a pass rate p over n independent cases — a quick sanity check, not a full analysis.

Is version B really better?



Habits that prevent false conclusions

Run multiple trials

Outputs vary run to run. Repeat each case (or fix sampling settings) and average.

Compare on the same cases

Paired comparisons — wins, losses, ties per case — detect smaller real differences than comparing two averages.

Watch the small slices

A slice of 20 cases can swing ±20 points by chance. Flag, don't panic.

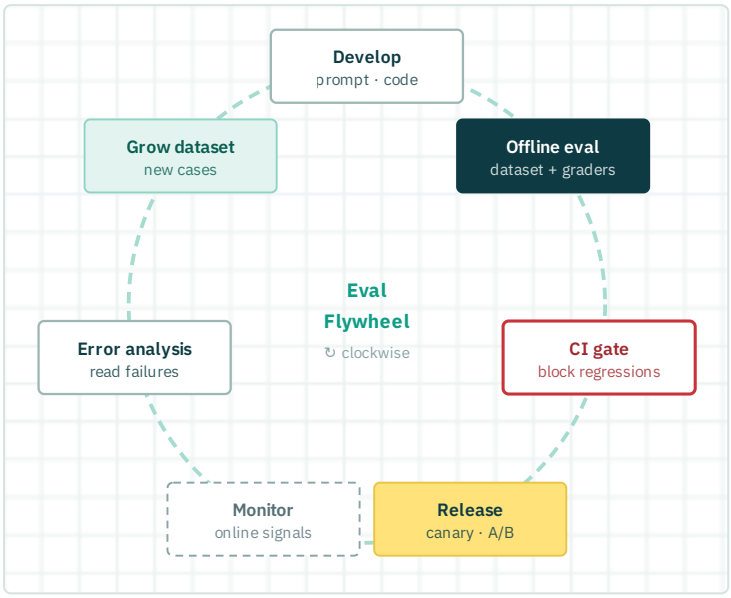
Decide thresholds first

Agree on “ship if ≥ X with no slice dropping > Y” before you see the numbers.

A 2-point gain on 100 cases is usually noise. Before celebrating an improvement or rolling back a regression, ask whether the difference is larger than the uncertainty in the measurement.

Evals Across the Lifecycle

Evals are not a one-time launch check. They run in development, gate releases in CI, and continue in production — where real failures feed the next round of tests.



In CI

Run fast code checks on every commit and the judged suite on every prompt, model or retrieval change. Fail the build on regressions beyond agreed thresholds.

At release

Roll out gradually. Compare the new version with the old on live traffic before switching everyone over.

In production

Monitor continuously, sample traces for review, and send every new failure back into the dataset.

	Offline evals	Online evals
Data	Fixed, curated dataset	Live production traffic
When	Before release, in CI	After release, continuously
Signals	Graded scores vs expected	Feedback, retries, sampled judges, A/B metrics
Catches	Regressions on known cases	New inputs, drift, real-world impact

Online signals

Explicit feedback

Thumbs, ratings, corrections.

Implicit signals

Retries, rephrasing, abandonment, escalations.

Sampled judging

Run LLM judges on a sample of live traces.

A/B tests

Compare versions on real users and business metrics.

Evals are not guardrails. Guardrails run on every request to block bad outputs in real time. Evals measure quality across many requests to guide decisions. Production systems need both — and guardrails need evals of their own.

Public Benchmarks vs Product Evals

Public benchmarks compare models on standard tasks. They are useful for orientation — but they measure someone else's problem, not yours.

Skill	Examples	What it tests
Knowledge	MMLU	Multiple-choice questions across many subjects
Expert reasoning	GPQA	Hard graduate-level science questions
Maths	GSM8K · MATH	Word problems to competition maths
Coding	HumanEval · SWE-bench	Functions from docstrings · real GitHub issues
Agents & tools	τ-bench	Multi-turn tool use with simulated users
Human preference	LMArena (Chatbot Arena)	Crowd-sourced pairwise votes between models

Why benchmark scores mislead

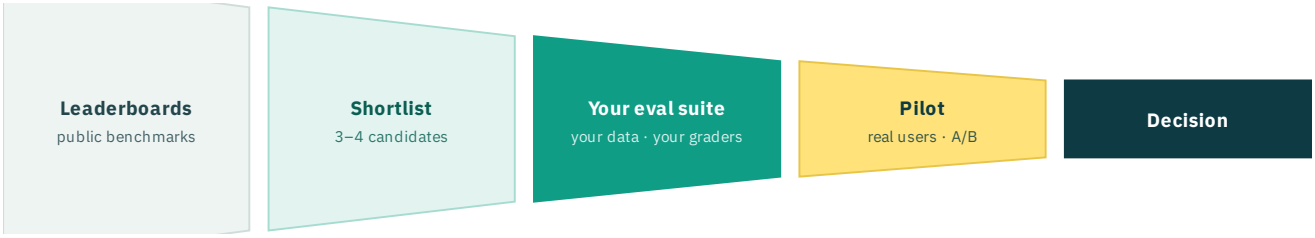
Contamination
Test questions leak into training data, inflating scores.

Saturation
Top models score near the ceiling; differences stop meaning much.

Mismatch
Benchmark tasks and formats rarely look like your product.

Goodhart's law
"When a measure becomes a target, it ceases to be a good measure."

Use benchmarks to shortlist — let your evals decide



What benchmarks are good for
Tracking the field, comparing models on general skills, and narrowing a long list of candidates cheaply.

What only product evals can tell you
Whether a model handles your prompts, tools, data, languages and edge cases — at a cost and latency you can afford.

A higher leaderboard rank is a hypothesis, not a result. Re-run your own eval suite whenever you consider switching models — the winner on a benchmark is often not the winner on your task.

Common Eval Pitfalls

Eight ways eval programmes go wrong — each with the symptom you will notice, the usual cause and how to fix it.

symptom ↗ root cause ↗ fix Most eval failures are process failures, not tooling failures.

Vibes only

Symptom Changes ship because a few examples “look better”.

Root cause No dataset or baseline.

Fix Start with 20–50 real cases and score every change.

Generic metrics

Symptom Scores move, but users notice no difference.

Root cause Metrics chosen from a list, not from failures.

Fix Do error analysis; build evals for real failure modes.

Unvalidated judge

Symptom LLM-judge scores disagree with what experts see.

Root cause Judge prompt never compared with human labels.

Fix Label a sample; measure agreement on passes and fails.

Overfitting the eval

Symptom Eval scores climb; production quality doesn't.

Root cause Prompt tuned on the same cases it is scored on.

Fix Keep a held-out set; refresh with new production cases.

Stale dataset

Symptom All green in CI, yet new complaints keep arriving.

Root cause Dataset no longer reflects real traffic.

Fix Add production failures continuously; retire old cases.

Averages only

Symptom Overall score up; one customer segment broken.

Root cause No per-slice reporting.

Fix Tag cases; report and gate on slices.

Ignoring cost & latency

Symptom Quality win, but the bill doubles and replies slow down.

Root cause Only quality was measured.

Fix Track tokens, cost and latency in every eval run.

Noise as signal

Symptom Teams chase 1–2 point swings between runs.

Root cause Small datasets, single trials, no intervals.

Fix Repeat trials, use paired comparisons, report uncertainty.

The best eval is the one your team actually runs. A small, trusted suite that runs on every change beats an elaborate one that runs once a quarter.

AI Evals Checklist

A one-page review for design docs, launch readiness and interviews. Not every system needs every item — but each should be a conscious decision.

Eval depth depends on Risk level · Task type · Output format · Release speed · Budget · Team size

Define 3

- [ ] Success criteria agreed with domain experts
- [ ] Error analysis on real traces
- [ ] Named failure modes to target

Dataset 4

- [ ] Cases from production + experts + synthetic
- [ ] Tags for slices and difficulty
- [ ] Held-out set kept separate
- [ ] Versioned and refreshed regularly

Graders 4

- [ ] Code checks for objective rules
- [ ] LLM judges with specific binary criteria
- [ ] Judges validated against human labels
- [ ] Human review for high-stakes cases

Analysis 4

- [ ] Per-slice reporting
- [ ] Multiple trials for non-determinism
- [ ] Confidence intervals / paired comparisons
- [ ] Cost and latency tracked

Operations 5

- [ ] Fast suite in CI on every change
- [ ] Release gates with agreed thresholds
- [ ] Online monitoring and sampled judging
- [ ] Production failures → new test cases
- [ ] Safety and red-team suite

How to use this list

Week one: read 100 traces, write 30 cases and a few code checks. Then add a validated judge, slices and CI gates as the product matures. Depth should match the risk of being wrong.

AI Evals on One Page

Eight steps from reading your first traces to a running eval flywheel — with the metric that tells you each one is working.

Understand

1 Analyze — Read traces → Annotate → Failure taxonomy
Key metric: failure modes ranked

2 Define — Success criteria → Binary checks → Thresholds
Key metric: criteria per mode

Build and score

3 Collect — Production → Experts → Synthetic
Key metric: coverage per slice

4 Grade — Code → LLM judge → Human
Key metric: judge–human agreement

5 Measure — Slices → Trials → Intervals
Key metric: pass rate ± CI

Operate

6 Gate — CI suite → Compare baseline → Ship / block
Key metric: regressions caught

7 Monitor — Feedback → Sampled judging → A/B
Key metric: online quality trend

8 Improve — Triage → New cases → Re-run
Key metric: time from failure to test

Benchmark ≠ product eval
Benchmarks shortlist models; your evals decide.

Offline ≠ online
Offline catches regressions; online catches the unknown.

Eval ≠ guardrail
Evals guide decisions; guardrails block in real time.

Without evals, every change is a guess.
With them, AI development becomes engineering.